

Contents

Tutorial: building an MLB dashboard in LibreOffice Calc	1
1. What this add-in does	1
2. Prerequisites	1
3. Install the extension	2
4. Configure your API key	2
5. Sanity check: your first formula	2
6. Build a small dashboard	3
6a. Look up one team's field	3
6b. That team's games (array formula)	3
6c. Standings	3
6d. Find a player and pull their season stats	3
6e. A single-game stat line	3
6f. Diagnostics	4
7. Recalculating	4
8. Understanding caching, errors, tiers, and rate limits	4
9. Using a per-cell API key instead of the environment	4
10. Where to go next	5

Tutorial: building an MLB dashboard in LibreOffice Calc

← Back to the [project README](#) (function reference and overview).

This is a hands-on walkthrough for the balldontlie MLB Calc add-in. The [README](#) is the function reference and [INSTALL.md](#) is the build guide — this document instead walks you, step by step, from “nothing installed” to a working spreadsheet that pulls live MLB data.

You'll build a small dashboard (a team lookup, that team's games, a player search, a player's season stats, and a single-game stat line) using every function the add-in exposes.

1. What this add-in does

It adds eleven MLB* worksheet functions to Calc, each backed by the [balldontlie](#) API. Type a formula like `=MLBTEAM(1; "abbreviation")` into a cell and, a second or two later, it resolves to real MLB data — no macros, no external app, just formulas.

Three things make it behave differently from a normal function:

- **It never blocks and never errors your sheet.** A fresh request shows `#FETCHING` while it fetches in the background; recalculate (**F9**) a moment later and the real value appears.
- **Table-shaped functions “spill” only via array entry.** Functions like `MLBTEAMS` or `MLBGAMES` return multiple rows/columns. LibreOffice has no dynamic spill (unlike Excel 365), so you select the output range first and confirm with **Ctrl+Shift+Enter**.
- **Scalar lookups take a field argument.** Instead of one function per piece of data, `MLBTEAM`, `MLBPLAYER`, `MLBGAME`, `MLBSEASONSTATS`, and `MLBSTAT` take the exact balldontlie JSON key you want as a string, e.g. `=MLBPLAYER(569; "full_name")`. The README's field-name reference lists every key you can ask for, including dot-notation for nested ones like `"home_team_data.runs"`.

Keep those three behaviors in mind — most “it's not working” moments trace back to one of them.

2. Prerequisites

- LibreOffice Calc (any recent version).

- A free balldontlie API key: sign up at <https://www.balldontlie.io/>. Without one, every MLB* data function shows #NO_API_KEY.
- The .oxt extension file itself — either downloaded prebuilt, or built from source (see step 3).

3. Install the extension

Easiest path — a prebuilt MLB.oxt:

```
mkdir -p ~/.config/libreoffice-mlb
echo 'api.key=your_key' > ~/.config/libreoffice-mlb/balldontlie.properties
"$LO_HOME/program/unopkg" add MLB.oxt
```

You can also just double-click the .oxt file to open LibreOffice's Extension Manager and install it that way.

Building it yourself is only necessary if you want to modify the Java source. It requires a JDK 8+ and the LibreOffice SDK; the full procedure (with platform-specific notes) is in [INSTALL.md](#). The short version:

```
export JAVA_HOME=~/.opt/jdk8
export LO_HOME=/usr/lib64/libreoffice
./build.sh
"$LO_HOME/program/unopkg" add --force build/MLB.oxt
```

Either way, **restart LibreOffice** after installing.

4. Configure your API key

You have four ways to supply the key (full priority order in the README); the properties file is the most convenient because it doesn't depend on how you launch LibreOffice:

```
mkdir -p ~/.config/libreoffice-mlb
cat > ~/.config/libreoffice-mlb/balldontlie.properties <<'EOF'
api.key=your_real_key_here
EOF
```

(macOS: use ~/Library/Application Support/libreoffice-mlb/ instead of ~/.config/libreoffice-mlb/.)

Restart LibreOffice so it picks up the file. If you'd rather not touch config files, you can instead type the key into a cell (e.g. B1) and pass `=$B$1` as the trailing `api_key` argument on every formula — useful for demo workbooks you plan to share, since the key then travels with the sheet only if you choose to fill that cell in.

5. Sanity check: your first formula

Open a new Calc document and type:

```
=MLBTEAMS()
```

Select a small range first (say A1:F5) and confirm with **Ctrl+Shift+Enter** rather than plain Enter, since this is a table-shaped function. You'll likely see #FETCHING in the top-left cell for a second — that's the background fetch kicking off. Press **F9** (recalculate) after a couple of seconds; the range should fill in with real team rows (id, abbreviation, display_name, location, league, division) — for example, row one resolving to something like 1, "ARI", "Arizona Diamondbacks", "Arizona", "National", "West".

If instead you see:

- #NAME? → the extension isn't registered. Run `unopkg list` and confirm `com.example.mlb` is present; reinstall if not.
- #NO_API_KEY → the key didn't resolve. Recheck step 4, and make sure you restarted LibreOffice after creating the properties file.
- #ERR → call `=MLBLASTERROR()` in an empty cell for the detail message.

6. Build a small dashboard

6a. Look up one team's field

Once you know a team's numeric id (from `MLBTEAMS()` above — team id 1 is the Arizona Diamondbacks, "ARI"), pull a single field with `MLBTEAM`:

```
A1: Team id      B1: 1
A2: Abbreviation B2: =MLBTEAM(B1; "abbreviation")
A3: Display name B3: =MLBTEAM(B1; "display_name")
```

`MLBTEAM` (and every other scalar function) returns whatever type the field actually is — a string here — so B2 resolves to `ARI` directly, no quotes, ready to use in another formula.

6b. That team's games (array formula)

```
A5: =MLBGAMES("2024"; B1)
```

Select several rows/columns below it (id, date, away_team_name, away_runs, home_team_name, home_runs, status, season_type) and enter with **Ctrl+Shift+Enter**. The second argument is the numeric team id from 6a — it's applied server-side, so it works whether you pass a date ("2024-06-01") or a season ("2024") as the first argument.

Once you spot a `game_id` in the results (column A), pull a single field from it with `MLBGAME`:

```
A20: Game status  B20: =MLBGAME(A5; "status")
```

(pointing at whatever cell holds a real game id from your `MLBGAMES` spill — e.g. game 21770 resolves "status" to "STATUS_FINAL").

6c. Standings

```
A22: =MLBSTANDINGS("2024")
```

Another array formula — spills team, league, division, wins, losses, win_percent, games_behind, run_differential.

Note: `/standings`, `/season_stats`, and `/stats` are **paid-tier** `balldontlie` endpoints. On a free key, `MLBSTANDINGS`, `MLBSEASONSTATS`, and `MLBSTAT` will show `#TIER` (`MLBLASTERROR()` → ...HTTP 401: Unauthorized) until you upgrade. `MLBTEAMS`, `MLBTEAM`, `MLBPLAYERSEARCH`, `MLBPLAYER`, `MLBGAMES`, and `MLBGAME` all work on the free tier — the rest of this tutorial's steps below need a paid key to actually resolve, but will still enter correctly as formulas.

6d. Find a player and pull their season stats

```
A24: Player search  A25: =MLBPLAYERSEARCH("Judge")
```

Array formula again — spills id/full_name/position/team/active, up to `max_rows` matches (100 by default). Once you spot the player you want (e.g. Aaron Judge, id 569, NYY):

```
A27: Player id      B27: 569
A28: Full name      B28: =MLBPLAYER(B27; "full_name")
A29: Season batting avg B29: =MLBSEASONSTATS(B27; "2024"; "batting_avg")
```

`MLBPLAYER` accepts dot-notation for nested fields too, e.g. `=MLBPLAYER(B27; "team.abbreviation")` pulls the abbreviation straight out of the player's nested `team` object.

6e. A single-game stat line

Given a player id and a game id (from 6b/6d):

```
A31: Hits in that game  B31: =MLBSTAT(B27; A5; "hits")
```

field here is whatever key balldontlie's `/stats` response uses for that player's line in that game — there's no fixed list, so experiment, or pull `MLBLASTERROR()` if a field name comes back `#NOT_FOUND`.

6f. Diagnostics

Two housekeeping formulas, handy while you're building:

`=MLBLASTERROR()` -> "" normally, or the detail behind the most recent `#ERR/#TIER/#RATE_LIMIT`
`=MLBCACHECLEAR()` -> clears the shared cache; returns the count cleared

`MLBCACHECLEAR()` is useful once you've fixed a bad key or want to force a fresh pull instead of waiting out the TTL (see step 8).

7. Recalculating

Because everything resolves asynchronously against a background cache, `#FETCHING` cells need a nudge to update once the fetch completes:

- **F9** recalculates the active sheet.
- **Ctrl+Shift+F9** force-recalculates everything — use this if F9 doesn't seem to clear a `#FETCHING`, since some dependent formulas need a second pass after the one that triggered the fetch resolves.

Expect the very first uncached formula in a session to take a couple of seconds; if you enter several *different* fresh requests back to back, each one can take up to ~13 seconds due to the free-tier rate limit (see below).

8. Understanding caching, errors, tiers, and rate limits

Cell shows	Meaning
<code>#FETCHING</code>	First request for this exact query. Recalculate shortly.
<code>#NO_API_KEY</code>	No key resolved — see step 4.
<code>#NOT_FOUND</code>	Reached the API, nothing matched (bad id, empty search, unknown field name).
<code>#TIER</code>	That endpoint (<code>MLBSTANDINGS/MLBSEASONSTATS/MLBSTAT</code>) needs a balldontlie plan above the free tier.
<code>#RATE_LIMIT</code>	Hit the free tier's ~5 requests/minute cap. Clears on its own after a short cooldown — recalculate again shortly.
<code>#ERR</code>	Persistent fetch failure. Check <code>MLBLASTERROR()</code> .

Responses are cached in memory for the life of the LibreOffice session, TTL depending on how often the underlying data changes: teams 24h, players 6h, games 5min, standings/season stats 1h, single-game stats 5min. You keep seeing the last-known-good value while a stale entry refreshes silently in the background — call `MLBCACHECLEAR()` to force a clean pull.

The free API tier allows roughly **5 requests/minute**. The add-in throttles itself to match (~13s between outgoing requests) and retries 5xx responses with backoff. A 429, though, isn't retried inline — it surfaces as `#RATE_LIMIT` immediately so one rate-limited call never snowballs into a burst of retries; normal use rarely hits the limit, but a burst of several *different* fresh formulas will visibly queue up.

9. Using a per-cell API key instead of the environment

Every data function takes an optional trailing `api_key` argument, which overrides whatever the environment resolves to for that one call:

```
=MLBTEAM(1; "abbreviation"; $B$1)  
=MLBSEASONSTATS(569; "2024"; "batting_avg"; $B$1)
```

Prefer a cell reference over typing the key literally into a formula, so it isn't spelled out (and duplicated) in every cell that uses it — this is also how you'd keep a shared workbook safe to hand off, with a placeholder key cell rather than a real key baked into formulas.

10. Where to go next

- **Full function reference** (every signature, every return shape, and the complete field-name reference for the `field` argument): the tables at the top of [README.md](#).
- **Behavior details** worth knowing before you build something serious — why field lookups return native types, how `MLBGAMES`'s team filter works in both date and season mode, `CompatibilityName` XLS/XLSX round-tripping: see “Behavior notes” in the README.
- **Building from source / troubleshooting the build:** [INSTALL.md](#).